



Testing Java classes with JUnit

(adapted from COS6028-B Software Systems Design and Testing)

Dr Daniele Scrimieri

COS6012-B Concurrent and Distributed Systems

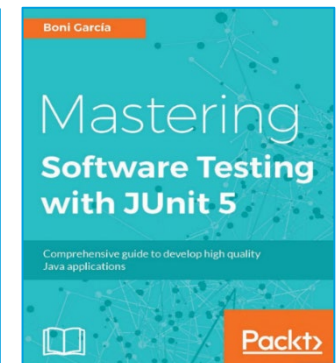
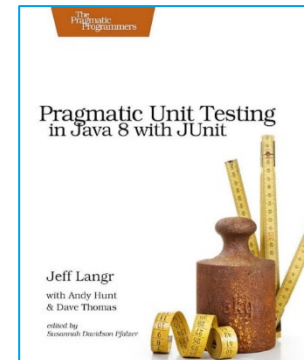
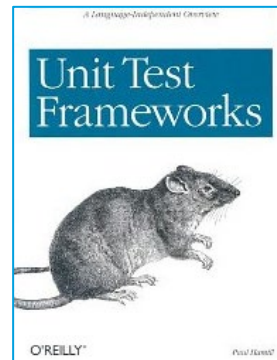
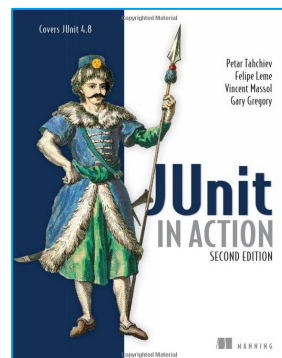
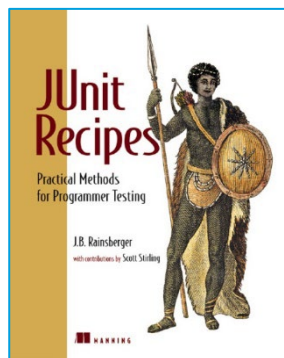
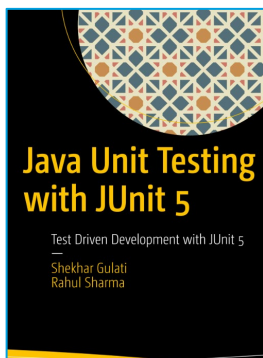
Outline

- Introduction to JUnit
- Step-by-step examples (from SSDT lab session)
 1. JUnit tests for Account class
 2. JUnit tests for Complex class
 - Overriding toString(), equals(Object) to support testing
 - Using setUp() and tearDown()

In this session you will learn...

- How to develop automated test code in JUnit, editing the auto-generated JUnit test method stubs for:
 - adding input values for tests
 - instantiating the objects
 - adding more method calls
 - adding more assert statements
 - writing additional test cases
- Other useful tips such as:
 - overriding **toString()**
 - overriding **equals(Object)**
 - using **setUp()** and **tearDown()**
- How to write JUnit tests for simple Java classes

- The main testing framework that we will use: www.junit.org
- JUnit is a popular unit testing framework for the Java programming language
- Very important in test-driven development
- Member of a family of unit testing frameworks which is collectively known as **xUnit** (www.wikipedia.org/wiki/XUnit), e.g., NUnit for .Net, PHPUnit for PHP, CppUnit for C++, etc.
- Integration with many IDEs: Eclipse, NetBeans, IntelliJ, etc.



- Used to write and run repeatable **automated tests**
- A structure for writing **test drivers**
- JUnit features include:
 - Assertions for testing expected results
 - Test features for sharing common test data
 - Test suites for easily organizing and running tests
 - Graphical and textual test runners
- JUnit is widely used in industry
- JUnit can be used as stand alone Java programs (from the command line) or within an IDE such as NetBeans or Eclipse

JUnit tests

- JUnit can be used to test
 - an entire object
 - part of an object – a method or some interacting methods
 - interaction between several objects
- It is primarily intended for unit and integration testing, not system testing
- Each test is embedded into one **test method**
- A **test class** contains one or more test methods
- Test classes include :
 - A collection of test methods
 - Methods to **set up** the state before and update the **state after** each test and before and after all tests

Writing tests for JUnit

- Need to use the methods of the **junit.framework.assert** class
- Each test method checks a condition (**assertion**) and reports to the test runner whether the test **failed** or **succeeded**
- The test runner uses the result to report to the user (in command line mode) or update the display (in an IDE)
- All of the methods **return void**
- A few representative methods of junit.framework.assert
 - assertTrue (boolean)
 - assertTrue (String, boolean)
 - fail (String)
 - fail("The test case is a prototype.");

Simple JUnit Example

```
public class Calc
{
    static public int add (int a, int b)
    {
        return a + b;
    }
}
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;
```

```
public class MyFirstJUnitJupiterTests {
{
    @Test
    void testAdd()
    {
        assertEquals (5, Calc.add (2, 3));
    }
}
```

Test
values

A blue oval containing the text 'Test values' has a blue arrow pointing from it to the '5' in the assertEquals call of the testAdd method in the MyFirstJUnitJupiterTests class.

Expected
output

A teal oval containing the text 'Expected output' has a teal arrow pointing from it to the '5' in the assertEquals call of the testAdd method in the MyFirstJUnitJupiterTests class.

Assertions available in JUnit

- `assertEquals(expected, actual)`
- `assertEquals(message, expected, actual)`
- `assertEquals(expected, actual, delta)` - used on doubles or floats, where delta is the allowable difference in precision
- `assertEquals(message, expected, actual, delta)` - used on doubles or floats, where delta is the allowable difference in precision
- `assertFalse(condition)`
- `assertFalse(message, condition)`
- `assertNotNull(object)`
- `assertNotNull(message, object)`

Assertions available in JUnit

- `assertNotSame(expected, actual)`
- `assertNotSame(message, expected, actual)`
- `assertNull(object)`
- `assertNull(message, object)`
- `assertSame(expected, actual)`
- `assertSame(message, expected, actual)`
- `assertTrue(condition)`
- `assertTrue(message, condition)`
- `fail()` - Fails a test with no message.
- `fail(message)` - Fails a test with the given message.
- `failNotEquals(message, expected, actual)`
- `failNotSame(message, expected, actual)`
- `failSame(message)`

Example 1: Account.java class

```
public class Account {  
  
    String holder;  
    double balance;  
  
    public Account(String name) {  
        holder = name;  
        balance = 0.0;  
    }  
  
    public void deposit(double amount) {  
        if (amount > 0)  
            balance += amount;  
    }  
}
```

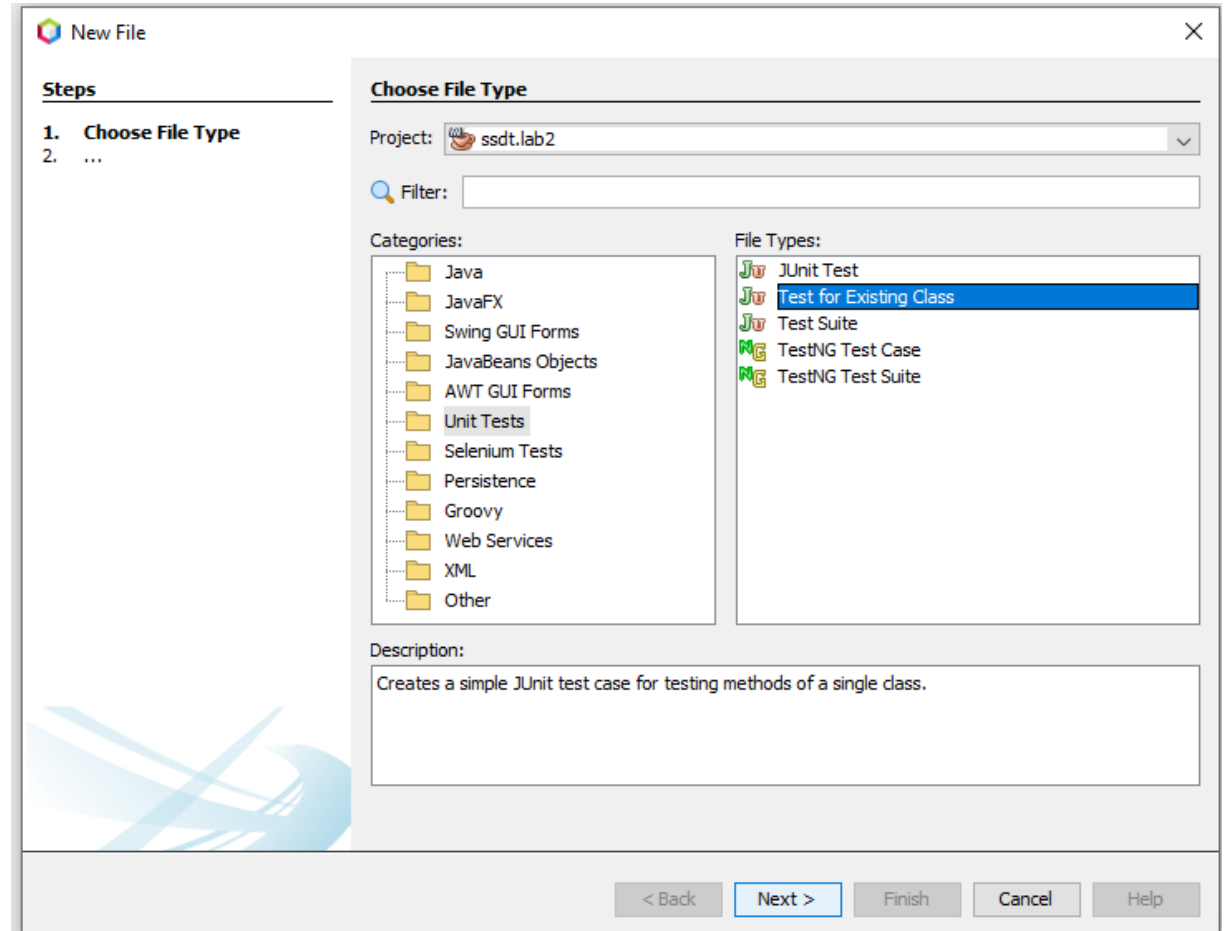
Example 1: Account.java class

```
public double withdraw(double amount) {  
    if (amount > 0 && balance >= amount) {  
        balance -= amount;  
        return amount;  
    } else  
        return 0;  
}  
  
public double getBalance() {  
    return balance;  
}  
  
public String getHolder() {  
    return holder;  
}  
  
}
```

Creating a new JUnit test in NetBeans

File -> New ->
Unit Tests ->
**Test for existing
class -> ...**

Choose the
Account class



Creating your first JUnit test...

- You can create JUnit tests in **NetBeans** for your classes with
 - File -> New File -> Unit Tests -> **Test for Existing Class** -> ... or
 - Tools -> **Create/Update Tests**
- **Eclipse**: New -> **JUnit Test case** and then choose the **Class under test**
- **IntelliJ**: Alt+Enter, and select **Create Test**
- Then you should fill in the automatically generated **method stubs** with appropriate assertions and input data
- With NetBeans you get a bit more help from the IDE: the method stubs have more content (method calls, assertions, etc.)

JUnit test class (automatically generated by NetBeans)

```
public class AccountTest {  
  
    @BeforeAll  
    public static void setUpClass () {  
    }  
  
    @AfterAll  
    public static void tearDownClass () {  
    }  
  
    @BeforeEach  
    public void setUp () {  
    }  
  
    @AfterEach  
    public void tearDown () {  
    }  
}
```

JUnit test class (stub automatically generated)

```
/**
 * Test of deposit method, of class Account.
 */
@Test
public void testDeposit() {
    System.out.println("deposit")
    double amount = 0.0;
    Account instance = null;
    instance.deposit(amount);
    // TODO review the generated test code
    // and remove the default call to fail.
    fail("The test case is a prototype.");
}
```

No **assert**, as
method
deposit(amount)
returns void

testWithdraw (stub automatically generated)

@Test

```
public void testWithdraw() {  
    System.out.println("withdraw");  
    double amount = 0.0;  
    Account instance = null;  
    double expectedResult = 0.0;  
    double result = instance.withdraw(amount);  
    assertEquals(expectedResult, result, 0.0);  
    // TODO review the generated test code  
    // and remove the default call to fail.  
    fail("The test case is a prototype.");  
}
```

testGetBalance (stub automatically generated)

@Test

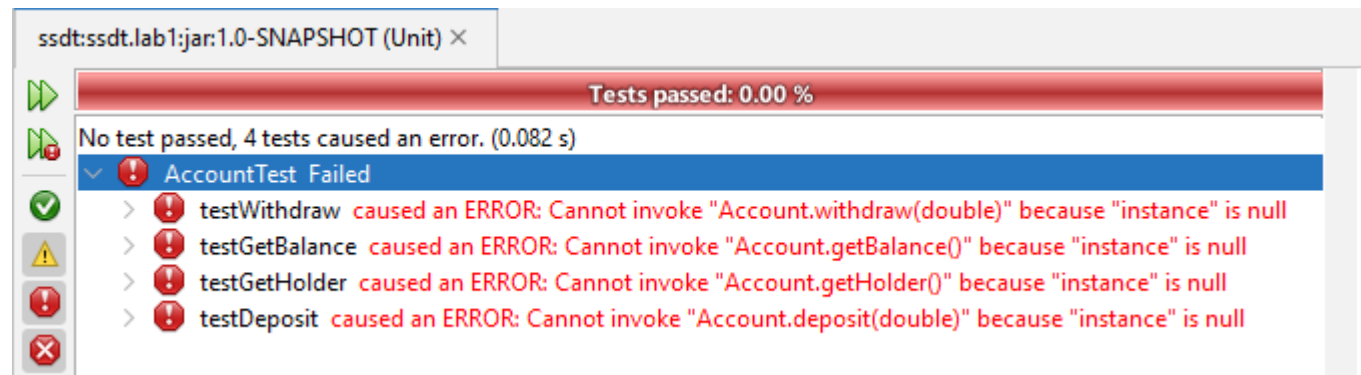
```
public void testGetBalance() {  
    System.out.println("getBalance");  
    Account instance = null;  
    double expectedResult = 0.0;  
    double result = instance.getBalance();  
    assertEquals(expectedResult, result, 0.0);  
    // TODO review the generated test code  
    // and remove the default call to fail.  
    fail("The test case is a prototype.");  
}
```

testGetHolder (stub automatically generated)

@Test

```
public void testGetHolder() {  
    System.out.println("getHolder");  
    Account instance = null;  
    String expResult = "";  
    String result = instance.getHolder();  
    assertEquals(expResult, result);  
    // TODO review the generated test code and  
    //remove the default call to fail.  
    fail("The test case is a prototype.");  
}
```

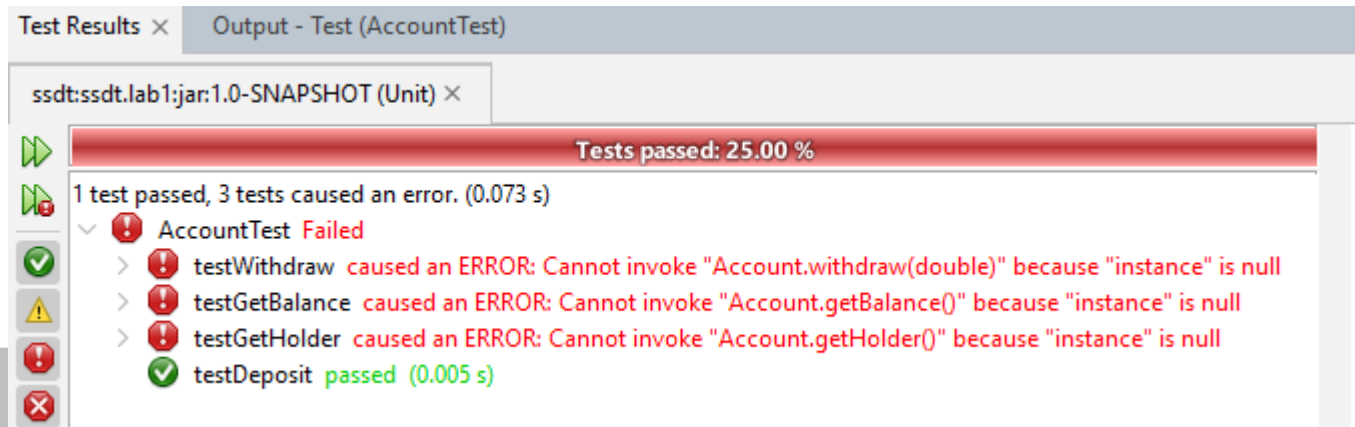
If you run the JUnit
AccountTest (right
click -> Test File):



Completing the JUnit tests...

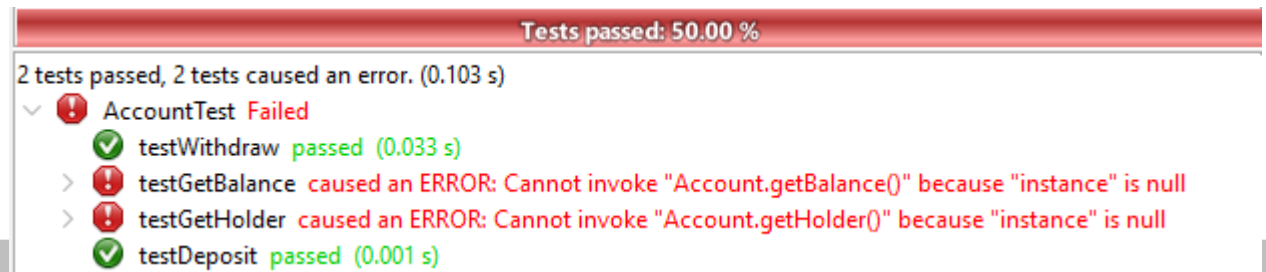
```
/**
 * Test of deposit method, of class Account.
 */
@Test
public void testDeposit() {
    System.out.println("deposit");
    double amount = 20.0;
    Account instance = new Account("John");
    instance.deposit(amount);
    assertEquals(20.0, instance.getBalance(), 0.0);
    // TODO review the generated test code
    // and remove the default call to fail.
    // fail("The
}
```

Added value for
amount,
instantiated a new
Account, added
assertEquals using
getBalance()



Completing JUnit testWithdraw method...

```
@Test
public void testWithdraw() {
    System.out.println("withdraw");
    Account instance = new Account("Alice") ;
    instance.deposit(100.0);
    double amount = 20.0;
    double result = instance.withdraw(amount);
    double expectedResult = 20.0;
    assertEquals(expResult, result, 0.0);
    assertEquals(80.0, instance.getBalance(), 0.0);
    // TODO review the generated test code
    // and remove the default call to fail.
    // fail("The test case is a prototype.");
}
```



Activity 1

- Continue completing the other 2 test methods for Account
- Add more tests for the Account class – one test per method is not enough!
- For example, you might want to:
 - Test **deposit** with a negative amount
 - Test **withdraw** with a negative amount
 - Test **withdraw** with an amount > balance

Adding a few more test methods...

```
@Test
public void testDepositNegative() {
    System.out.println("test deposit Negative");
    double amount = -30.0;
    Account instance = new Account("Ali");
    instance.deposit(amount);
    assertEquals(0.0, instance.getBalance(), 0.0);
}
```

```
@Test
public void testWithdrawNotAccepted() {
    System.out.println("test withdraw Not Accepted");
    Account instance = new Account("Sarah");
    instance.deposit(100.0);
    double amount = 200.0;
    double result = instance.withdraw(amount);
    double expectedResult = 0.0;
    assertEquals(expectedResult, result, 0.0);
    assertEquals(100.0, instance.getBalance(), 0.0);
}
```

Another example: a class for **Complex** numbers

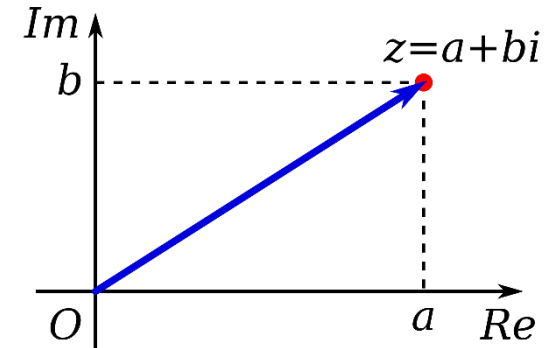
```
public class Complex {
```

```
    double re, im;
```

```
    public Complex() {  
        this.re = 0.0;  
        this.im = 0.0;  
    }
```

```
    public Complex(double real, double imaginary) {  
        this.re = real;  
        this.im = imaginary;  
    }
```

```
    public static Complex sum(Complex c1, Complex c2) {  
        return new Complex(c1.re + c2.re, c1.im + c2.im);  
    }
```



...

https://en.wikipedia.org/wiki/Complex_number

JUnit test method (stub automatically generated)

```
public class ComplexTest {  
    ...  
  
    /**  
     * Test of sum method, of class Complex.  
     */  
    @Test  
    public void testSum() {  
        System.out.println("sum");  
        Complex c1 = null;  
        Complex c2 = null;  
        Complex expResult = null;  
        Complex result = Complex.sum(c1, c2);  
        assertEquals(expResult, result);  
        // TODO review the generated test code and  
        // remove the default call to fail.  
        fail("The test case is a prototype.");  
    }  
}
```

JUnit test case – after completing it...

```
@Test
public void testSum() {
    System.out.println("sum");
    Complex c1 = new Complex(1.0, 2.0);
    Complex c2 = new Complex(3.0, 4.0);
    Complex expResult = new Complex(4.0, 6.0);
    Complex result = Complex.sum(c1, c2);
    assertEquals(expResult, result);
    // TODO review the generated test code
    // and remove the default call to fail.
    // fail("The test case is a prototype.");
}
```

If you run
ComplexTest



```
testSum Failed: expected: <ie.Complex@5e91993f> but was: <ie.Complex@1c4af82c>
expected: <ie.Complex@5e91993f> but was: <ie.Complex@1c4af82c>
junit.framework.AssertionFailedError
```



Overriding `toString()` method in our `Complex` class

```
public String toString() {  
    String s = "";  
    if (re != 0)  
        s = s + re;  
    if (im > 0)  
        s = s + "+" + im + "i";  
    else if (im < 0)  
        s = s + im + "i";  
    return s;  
}
```

Run now
Complex
Test:

  testSum Failed: expected: ie.Complex<4.0+6.0i> but was: ie.Complex<4.0+6.0i>
.....expected: ie.Complex<4.0+6.0i> but was: ie.Complex<4.0+6.0i>
.....junit.framework.AssertionFailedError

?

Overriding `equals(...)` method in our `Complex` class

```
public boolean equals(Object other) {  
    if (other == null) return false;  
    if (other instanceof Complex) {  
        Complex c = (Complex) other;  
        return (re==c.re && im==c.im);  
    }  
    else return false;  
}
```

Run now
ComplexTest:



testSum passed (0.001 s)

setUp, tearDown and test methods order

We can prepare the configuration and test objects, that will be used in the test methods, using the **setUp()** method.

The methods **setUp()** and **tearDown()** will be called for each test case method calls (before and after) for:

- preparation and post clean up process for each JUnit test call.

- setUpClass
 - setUp
 - testMethod1()
 - tearDown
 - setUp
 - testMethod2()
 - tearDown
 - [...]
 - setUp
 - testMethodN()
 - tearDown
- tearDownClass

Example of using setUp()

```
public class ComplexTest {
```

```
    Complex c1;
```

```
    Complex c2;
```

```
    ...
```

```
    @Before
```

```
    public void setUp() {
```

```
        System.out.println("setUp ");
```

```
        c1 = new Complex(2,3);
```

```
        c2 = new Complex(1,2);
```

```
    }
```

```
c1 = 2 + 3i  
c2 = 1 + 2i
```

Using the objects instantiated in setUp() other test methods

@Test

```
public void testSum() {  
    System.out.println("sum");  
    Complex expResult = new Complex(3, 5);  
    Complex result = Complex.sum(c1, c2);  
    assertEquals(expResult, result);  
}
```

$c1 = 2 + 3i$
 $c2 = 1 + 2i$

Expected sum:
 $c1 = 3 + 5i$

@Test

```
public void testDifference() {  
    System.out.println("difference");  
    Complex expResult = new Complex(1,1);  
    Complex result = Complex.difference(c1, c2);  
    assertEquals(expResult, result);  
}
```

Expected diff:
 $c1 = 1 + i$

Activity 2

- Create in your Java project the **Complex** class described
- Generate automatically the JUnit test stubs for it
- Provide input values and expected values in the test methods
- Follow the explanations from the slides and add code for the **toString()** and **equals()** methods in the Complex class
- Continue completing all the test methods for **ComplexTest**
- Add more tests in the **ComplexTest** class – only one test per method is not enough!

Other JUnit resources

- <https://junit.org/>
- JUnit and NetBeans:
<https://blogs.apache.org/netbeans/entry/junit-5-apache-ant-and>
- JUnit and Eclipse:
<http://www.vogella.com/tutorials/JUnit/article.html>
- JUnit and IntelliJ:
<https://www.jetbrains.com/help/idea/junit.html>
- Getting started with JUnit in NetBeans:
https://www.youtube.com/watch?v=Vnyeq3ZS_1Q&ab_channel=BrandonMyers